

一整个班的 40 个名字，就一个变量

第4讲 · 组合数据类型

列表与元组

这一讲，你会收获四件事

- 打包：列表 & 元组 怎么建、怎么用
- 可变 vs 不可变：一字之差，天壤之别
- 批量：切片、遍历、排序一套带走
- AI 落地：采样本、看损失、洗文本

第一幕 · 装进一组数据

从单人座位，到一排座位

```
score = 88          # 单个变量  
scores = [78, 88, 95, 66]  # 一个列表装多个分数
```



"一整个班的名字，怎么办？" → 交给列表

下标从 0 开始，负数从右数

```
nums    = [1, 2, 3, 4]      # 整数列表
names    = ["Alice", "Bob"] # 字符串列表
mix      = [1, "A", 3.14]   # 可混型列表
empty    = []               # 空列表

print(nums[0])      # 1      下标从 0 开始
print(names[-1])    # 'Bob'  负数下标从右数
print(len(nums))    # 4      元素个数
```



增删改齐了，列表才真的"活"

```
scores = [78, 88, 95]
scores[1] = 99      # 改：把下标1换成99
scores.append(100)   # 增：末尾追加一个元素
scores.remove(78)    # 删：删除值78（第一个匹配）
popped = scores.pop() # 弹：弹出并返回末尾元素
print(scores)        # [99, 95]
```



★ append / extend / insert: 三种"塞进去"

```
lst = [1, 2, 3]
lst.append(4)      # 塞单个    → [1,2,3,4]
lst2 = [1, 2, 3]
lst2.extend([4, 5]) # 展平拼    → [1,2,3,4,5]
lst3 = [1, 2, 3]
lst3.insert(1, 99) # 插中间    → [1,99,2,3]
```



- append 塞"一个", extend 展平"一串"
- insert 找准"位置", 可插中间
- ⚠ lst.append([4,5]) 是塞"整体" → [1,2,3,[4,5]]

采样本：一个 append 一个样本

```
dataset = []                                # 空数据集
for i in range(1, 6):
    dataset.append(f"img_{i:03d}.jpg")
print(dataset)
```



第二幕・整段取用

含头不含尾，一次记死

```
nums = [10, 20, 30, 40, 50]
print(nums[1:4])      # [20, 30, 40]    含头不含尾
print(nums[:3])       # [10, 20, 30]   从头到位置3
print(nums[::2])      # [10, 30, 50]   步长为2
print(nums[::-1])     # 反转整张列表
```



for: 最常用的批处理咒语

```
for name in ["Alice", "Bob", "Cathy"]:  
    print(name)
```

```
for i, name in enumerate(names):  
    print(i, name)
```

```
squares = [x * x for x in range(1, 6)]
```



sort() 原地动手，sorted() 返回新表

```
losses = [0.9, 0.4, 0.6, 0.2, 0.7]
losses.sort()           # 原地升序
print(losses)           # [0.2, ... , 0.9]

losses.sort(reverse=True) # 原地降序
sorted(losses)            # 返回新列表,原表不动
```



把损失排一排，收敛一眼见

```
epoch_losses = [1.2, 0.8, 0.6, 0.3, 0.2]
stable = sorted(epoch_losses)      # 不看顺序,看数值分布
print("最低损失:", stable[0])      # 0.2

epoch_losses.sort(reverse=True)    # 降序看收敛趋势
print(epoch_losses)
```



表格按下标，矩阵按行

```
grades = [  
    [78, 88, 95],  
    [66, 70, 80],  
    [90, 92, 99],  
]  
print(grades[0][1])      # 88 → 第0个学生的第1门课  
for row in grades:  
    print("该行总分:", sum(row))
```



第三幕·锁住不动的数据

不能改的，反而是最稳的

```
t = (1, 2, 3)
t2 = ("apple",)    # 单个元素必须有逗号!
print(t[0])        # 1
a, b, c = t         # 元组解包,一次拆三份
```



给数据上一把"锁"

```
lst = [1, 2, 3]
lst[0] = 99      # ✅ 列表可以改
print(lst)      # [99, 2, 3]

t = (1, 2, 3)
t[0] = 99        # ❌ TypeError: 'tuple' object ...
```



需要保护、不许被意外改动的数据 → 交给元组

一边遍历，一边清洗

```
raw_texts = ["Hello", "WORLD", "python"]
cleaned = []
for text in raw_texts:
    cleaned.append(text.lower())
print(cleaned)           # ['hello', 'world', 'python']

nouns = ("cat", "dog", "bird") # 只读常量表
```



五个坑，踩一个算我输

想干嘛	坑	对的做法
列表想接 [4,5]	<code>append([4,5])</code> → <code>[1,2,3,[4,5]]</code>	用 <code>extend([4,5])</code>
改元组	<code>t[0]=99</code> → <code>TypeError</code>	改用列表，或元组只读
下标越界	<code>nums[5]</code> 但只有 4 个	先看 <code>len</code> ，或遍历
写单元素元组	<code>t=(5)</code> 其实是 <code>int</code>	写 <code>t=(5,)</code>
保留原列表排序	<code>sort()</code> 原地改掉了	用 <code>sorted()</code>

结课：把"序列"这根线，留到下次

列表、元组、字符串，都会下标与切片——它们凭什么是一家？答案藏在"序列"两个字里。

➡
SOON 下节预告：集合 set 与字典 dict